

EVENT & PARKING RESERVATION SYSTEM

TECHNICAL STACK DOCUMENT

Technologies, versions, architecture, APIs, libraries, tools and team delivery workflow

Document Type	Technical Stack & Implementation Environment
Project Baseline	BRD v1.3 + approved integrated project structure
Architecture	.NET 8 Simple N-Tier / layered architecture
Team Context	6-member collaborative full-stack project
Prepared By	Group B1 – G5

1. Purpose and Version Accuracy

This document records the technology stack actually selected and used for the Event & Parking Reservation System. It separates project target versions from local-machine tool versions so that the team can reproduce the solution without accidentally changing the approved architecture.

Version accuracy rule: Where the project sources contain an exact version, it is stated. Where only the product/family is confirmed, the exact patch version is marked as not explicitly recorded rather than guessed.

2. Core Technical Stack - Executive Matrix

Area	Technology / Version	Evidence Status	Project Use
Application type	Full-stack web application	Confirmed	Customer + Administrator web system
Architecture	Simple N-Tier / layered	Confirmed	Api -> Business -> DataAccess -> Models; Tests separate
Target framework	.NET 8 / net8.0	Confirmed exact major	All backend projects target .NET 8
Backend framework	ASP.NET Core Web API	Confirmed	REST controllers, middleware, DI, authorization
Primary backend language	C#	Confirmed	Language version not explicitly pinned in source evidence
ORM	Entity Framework Core 8	Confirmed	Latest recorded project migration/tool version: 8.0.29
Database	Microsoft SQL Server / LocalDB	Confirmed	Exact SQL Server engine version not explicitly recorded
Frontend	HTML5 + CSS3 + Vanilla JavaScript	Confirmed	No frontend framework
Frontend API client	Fetch API	Confirmed	Browser-native API, no package version
API documentation/testing	Swagger / OpenAPI	Confirmed	Swashbuckle.AspNetCore recorded as 6.6.2
Authentication	JWT Bearer	Confirmed	Role-based Customer / Administrator authorization
Password hashing	BCrypt.Net-Next	Confirmed	4.2.0 recorded
Source control	Git + GitHub	Confirmed	develop integration branch + feature/fix branches + PRs

3. Backend Architecture and Project Structure

The final team architecture is a Simple .NET 8 N-Tier design. It is intentionally not Clean Architecture, Onion Architecture, CQRS or Microservices. The backend is split into five fixed projects:

Project	Layer	Responsibilities
EventParking.Api	ASP.NET Core presentation/API layer	Controllers, Swagger, JWT configuration, middleware, HTTP request/response pipeline
EventParking.Business	Business/application logic	DTOs, service interfaces, service implementations, validation/business rules
EventParking.DataAccess	Persistence layer	ApplicationDbContext, EF Core configurations, repositories, migrations, seed data
EventParking.Models	Shared domain model	Entities, enums, base/common models and cross-layer data structures
EventParking.Tests	Automated test project	Unit/controller/service tests and regression coverage

Dependency direction: EventParking.Api -> EventParking.Business -> EventParking.DataAccess -> EventParking.Models. EventParking.Tests references the projects required for testing. Circular references are not allowed.

4. Backend Runtime, Frameworks and NuGet Libraries

Component	Version	Status	Purpose / Note
.NET target	net8.0 / .NET 8	Confirmed	Project target framework. A member machine later showed .NET SDK 9.0.302 being used to compile net8.0; that SDK is an environment detail, not the project target.
ASP.NET Core Web API	8.x family via .NET 8	Confirmed major	Exact ASP.NET Core patch version not separately recorded.
Microsoft.EntityFrameworkCore	8.0.29 latest recorded	Confirmed	Migration history and EF CLI evidence record ProductVersion 8.0.29.
Microsoft.EntityFrameworkCore.SqlServer	8.0.29 latest recorded	Confirmed	SQL Server provider; earlier screenshot showed 8.0.15 before later package alignment.
Microsoft.EntityFrameworkCore.Design	8.0.29 latest recorded	Confirmed	Design-time migrations/scaffolding.
Microsoft.EntityFrameworkCore.Tools	8.0.29 latest recorded	Confirmed	Visual Studio/EF tooling package.
dotnet-ef	8.0.29 aligned environment	Confirmed later aligned state	A temporary 10.0.10 installation was observed during troubleshooting; project-compatible EF tooling was subsequently 8.0.29.
Swashbuckle.AspNetCore	6.6.2	Recorded	Swagger/OpenAPI generation and Swagger UI; no later package change was verified.
BCrypt.Net-Next	4.2.0	Confirmed	Password hashing.
Microsoft.AspNetCore.Authentication.JwtBearer	8.x compatible; patch not recorded	Confirmed use	Bearer token authentication in Program.cs/shared auth configuration.
Microsoft.IdentityModel.Tokens	Version not explicitly recorded	Confirmed use	JWT signing key and token validation primitives.
System.Text.Json	Built into .NET 8	Confirmed use	JSON serialization and enum/string JSON configuration.
Built-in Dependency Injection	ASP.NET Core .NET 8	Confirmed	Scoped repositories/services and framework services.
BackgroundService	ASP.NET Core .NET 8	Confirmed	BookingExpiryService periodically releases expired pending holds.
xUnit	Version not explicitly recorded	Confirmed from tests	Unit/controller test framework.
Moq	Version not explicitly recorded	Confirmed from tests	Mocking repositories/services in unit tests.

5. Frontend Technology Stack

Technology	Version / Standard	Use
HTML5	HTML5 standard	Semantic page structure for public, customer and admin pages
CSS3	CSS3 standard	Layout, components, responsive styling
Vanilla JavaScript	ECMAScript/browser JavaScript; exact language edition not pinned	DOM manipulation, state handling, validation, event handling
Fetch API	Browser-native; no package version	All frontend-to-backend HTTP communication
CSS Grid	Browser standard	Page and dashboard layout
Flexbox	Browser standard	Navigation, forms, cards and responsive alignment
Media Queries	Browser standard	Responsive behaviour across viewport sizes
LocalStorage	Browser Web Storage API	JWT/role/session-related client state in the implemented static frontend
Live Server / static server	Version not recorded	Local frontend serving during development, commonly port 5500

The frontend is a static multi-page application. The approved project explicitly avoids React, Angular, Vue, Next.js and other frontend frameworks. This keeps the implementation aligned with the BRD requirement for plain HTML/CSS/JavaScript using Fetch API.

6. Database and Persistence Stack

Area	Technology / Object	Use / Note
Database engine	Microsoft SQL Server	Exact engine version not recorded
Development database	SQL Server LocalDB	Swagger evidence records EventParkingReservationDb on LocalDB
ORM	Entity Framework Core 8.0.29	Code-first entity mapping, LINQ queries, persistence
DbContext	ApplicationDbContext	Single shared DbContext across modules
Migrations	EF Core Migrations	InitialCreate plus iterative schema changes for parking, facilities, food court, booking price snapshot, notification indexes, etc.
Constraints / indexes	SQL Server constraints and filtered/unique indexes	Protect data integrity and prevent conflicting active reservations
Transactions	EF Core / SQL Server transactions	Serializable transaction logic used in booking/expiry-sensitive flows
Seed data	EF Core seed/startup seed + project SQL data where used	Development/demo users, venues, categories, events and supporting records

7. API and Integration Technologies

The system exposes JSON REST APIs through ASP.NET Core controllers and consumes them from the static frontend with Fetch API. Swagger/OpenAPI is used for interactive documentation and manual integration testing.

API Group	Representative Route	Responsibility
Authentication	/api/auth	Register, verify email, resend verification, login, forgot/reset password
Customers	/api/customers	Profile, customer administration, activation/deactivation safeguards
Venues	/api/venues	Venue CRUD and availability
Categories	/api/categories	Event category CRUD
Events	/api/events	Event CRUD, search/filter, overlap/capacity rules
Seats	/api/events/{eventId}/seats	Seat map generation/listing/validation/update/delete

API Group	Representative Route	Responsibility
Parking	/api/events/{eventId}/parking-slots	Parking layout and availability; booking-integrated reservation
Bookings	/api/bookings	Create booking, hold state, history/cancellation lifecycle
Payments	/api/payments and booking payment routes	Simulated payment and payment history
Notifications	/api/notifications	List, unread count, mark-read / mark-all-read
Food Court	food-stalls / food-items / food-orders	Stall, menu, pre-order and order-status lifecycle
Facilities	/api/facilities / venue facility routes	Venue facility management and customer guidance
Dashboards	dashboard routes	Customer/admin operational summaries where integrated

7.1 HTTP and Security Conventions

- JSON request/response bodies over HTTP/HTTPS.
- Standard REST-style methods: GET, POST, PUT and DELETE.
- JWT Bearer token supplied in the Authorization header for protected endpoints.
- Role-based authorization separates Customer and Administrator actions.
- Common response statuses observed during testing include 200, 201, 400, 401, 404 and 409.
- CORS is configured so the static frontend can call the local ASP.NET Core API.
- Swagger UI is the primary manual API testing/exploration interface during development.

8. Development, Testing and Collaboration Tools

Tool	Primary Use	Version / Note
Visual Studio 2022	Backend development, debugging, NuGet, EF migrations	Project standard. Developer PowerShell evidence shows member environments such as v17.10.1 and v17.14.20; exact IDE patch varies by machine.
Visual Studio Code	Frontend HTML/CSS/JS editing	Version not recorded.
SQL Server Management Studio	Database inspection/administration	Version not recorded.
Swagger UI	Manual API testing	Provided through Swashbuckle/OpenAPI.
Git	Local version control	Version not recorded.
GitHub	Remote repository, Pull Requests and integration	Primary repo: rvabi/EventParkingReservationSystem; develop is the integration branch.
dotnet CLI	Restore, build, test, run, EF tooling	Project targets net8.0.
Browser DevTools	Frontend network/debug validation	Used with static frontend and API.
Antigravity / cloud AI assistant	Read-only analysis, implementation assistance, audits	Used under Second Brain rules; not part of runtime stack.

9. Technical Team Workflow and Integration Structure

The six-member team develops modules in parallel but integrates them through a shared architecture and Git workflow. The stack is therefore not only a list of technologies; it is also governed by a controlled integration process to protect shared files such as Program.cs, ApplicationDbContext, migrations and common frontend helpers.

#	Stage	Technical Gate
1	Start from updated develop	Each feature/fix branch is based on the integration branch.
2	Implement one focused module/task	Use existing projects/layers; do not create duplicate backend architecture.
3	Build and test	Run relevant .NET tests and verify business rules.

#	Stage	Technical Gate
4	Swagger/manual API validation	Verify routes, authorization, HTTP responses and database effects.
5	Frontend browser smoke test	For UI changes, validate Fetch requests, loading/error states and role access.
6	Review Git diff/status	Ensure only intended files changed and no secrets/build artifacts are committed.
7	Commit and push feature branch	Use clear type(scope): action commit messages.
8	Pull Request to develop	Team integration occurs through PR review/merge.
9	Full regression + clean clone	Team Lead validates integrated backend/frontend and setup reproducibility.
10	develop -> main release	Final stable integration proceeds to main and release tagging when ready.

Shared-file protection: Program.cs, ApplicationDbContext.cs, appsettings*.json, EF migrations/model snapshot, shared enums, .csproj/.sln and central frontend API/auth/config files require extra integration care because multiple modules depend on them.

10. Runtime Technology Flow

The main customer journey demonstrates how the stack works end-to-end:

- Browser renders HTML/CSS and runs Vanilla JavaScript.
- JavaScript calls the ASP.NET Core REST API using Fetch API.
- JWT Bearer authentication identifies the customer and role on protected routes.
- Controller delegates to the Business service layer.
- Business services enforce booking, pricing, capacity, status and ownership rules.
- DataAccess repositories/ApplicationDbContext query SQL Server using EF Core.
- Booking logic uses transactions/constraints to protect seats and optional parking from double allocation.
- Simulated payment confirms the booking; notifications are stored for the customer.
- BookingExpiryService releases expired pending resources in the background.
- Frontend refreshes API data and updates the customer/admin interface.

11. Version Traceability and Environment Notes

Technology	Recorded Version	Interpretation
.NET target	8 / net8.0	Current frozen project architecture and csproj/build paths
EF Core runtime/migrations	8.0.29	Recorded in __EFMigrationsHistory ProductVersion and EF CLI output
EF Core Design/SqlServer/Tools	8.0.29	Latest recorded package listing/tool evidence
BCrypt.Net-Next	4.2.0	Recorded package listing
Swashbuckle.AspNetCore	6.6.2	Recorded earlier csproj screenshot; later change not verified
Visual Studio	2022	Project standard; patch differs by developer machine
Local .NET SDK example	9.0.302 observed on one machine	Can build net8.0, but is not the project target
SQL Server	SQL Server / LocalDB	Exact engine version not preserved in available evidence
VS Code / SSMS / Git	Not explicitly recorded	Do not invent patch versions in formal submission

Recommended reproducibility rule: Keep the backend target at net8.0 and keep Entity Framework Core/dotnet-ef on the 8.x line (the latest project evidence shows 8.0.29). Do not upgrade runtime libraries merely because a developer PC has a newer SDK installed.

12. Explicit Technology Boundaries

The following technologies are not part of the approved project stack unless the team formally changes the architecture:

- React / Angular / Vue / Next.js frontend frameworks
- Node.js backend
- Microservices architecture
- MediatR / CQRS as a required architecture pattern
- Real payment gateway integration
- SignalR as a required real-time feature
- Docker as a mandatory local-development dependency
- Physical parking hardware integration
- Multiple databases or a second backend solution

13. Source Basis Used for This Technical Stack Document

Source	What it supports
Project BRD v1.3	Defines web-based system and suggested stack: ASP.NET Core Web API, SQL Server, HTML/CSS/JavaScript, Fetch API.
Master / Updated Second Brain	Freezes .NET 8 N-Tier architecture, five backend projects, frontend technology, database/auth/tooling and Git rules.
Backend Swagger Testing Documentation - 10 Aug 2026	Confirms .NET 8 API runtime, SQL Server LocalDB, JWT, Swagger integration and tested module groups.
Member integration evidence - 10 Aug 2026	Confirms static frontend/browser smoke testing and integrated venue/event/facility API usage.
Package / migration screenshots and logs - 6-11 Aug 2026	Provide exact package/tool evidence such as EF Core 8.0.29, BCrypt.Net-Next 4.2.0 and Swashbuckle 6.6.2.
Project chat and integrated workflow history	Provides team integration sequence, branch/PR workflow, shared-file controls and final regression/release process.

14. Final Technical Stack Summary

The Event & Parking Reservation System is a .NET 8 full-stack web application implemented with ASP.NET Core Web API, C#, Entity Framework Core 8 and SQL Server, with a plain HTML5/CSS3/Vanilla JavaScript frontend using the Fetch API. Authentication is JWT-based, passwords are hashed with BCrypt, Swagger/OpenAPI supports API exploration/testing, and the solution follows a fixed five-project N-Tier architecture. The latest recorded EF Core package/tooling version is 8.0.29, BCrypt.Net-Next is 4.2.0, and Swashbuckle.AspNetCore is recorded as 6.6.2. The project is delivered through Git/GitHub feature branches and Pull Requests into develop, followed by integrated regression, clean-clone validation and final release to main.

15. Frontend Implementation Visual Evidence

This section adds project-related frontend visuals to the Technical Stack Document. The visuals are browser renders based on the current develop frontend page structure and confirmed project integration/demo records. No unrelated stock artwork or newly invented UI concept is used.

15.1 Event Discovery Frontend

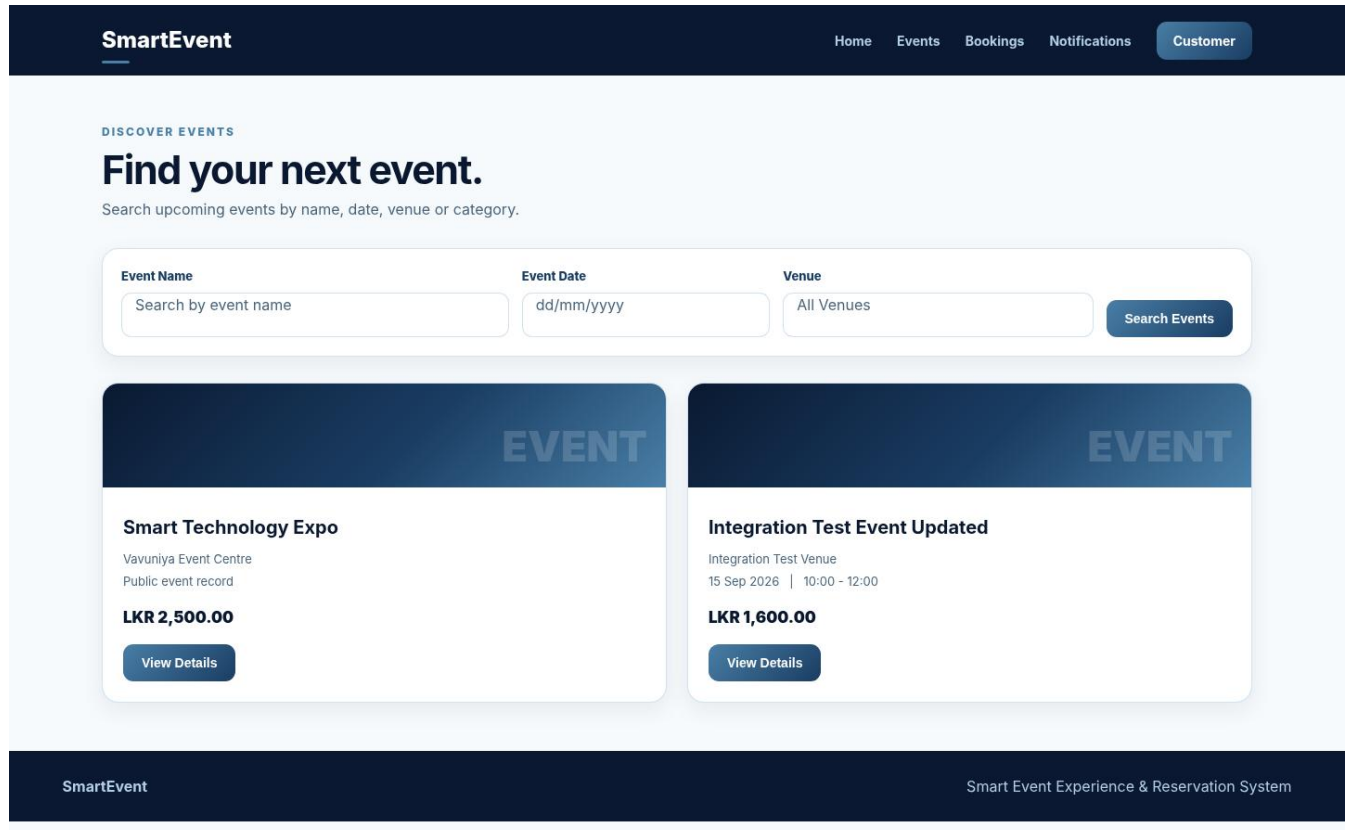


Figure 1 - Event Discovery / Events Page

Technical relevance: HTML5 structures the filters/cards, CSS3 provides the responsive visual system, Vanilla JavaScript loads event data, and Fetch API connects the page to the Events, Venues and Categories endpoints.

15.2 Seat Selection and Optional Parking Flow

SmartEvent

HomeEventsBookingsNotificationsCustomer

SEAT SELECTION

Smart Technology Expo

Vavuniya Event Centre | Ticket price LKR 2,500.00

Select your seats

Refresh

STAGE

A1A2A3A4A5A6A7A8A9A10

B1B2B3B4B5B6B7B8B9B10

C1C2C3C4C5C6C7C8C9C10

D1D2D3D4D5D6D7D8D9D10

Available

Selected

Held

Booked

Unavailable

Your Selection

A1LKR 2,500

Seat TotalLKR 2,500

✓ Seats are available

Parking (optional)

Choose parking if needed, then create your booking.

No Parking

Skip parking for this booking.

P01 · Zone A

AvailableLKR 500

Create Booking

Figure 2 - Customer Seat Selection with Optional Parking

Technical relevance: this screen demonstrates the frontend integration boundary between the Seats, Parking Slots and Bookings APIs. The UI is informative, while final resource availability and conflict protection remain enforced by the ASP.NET Core service/database transaction logic.

Prepared 11 August 2026 | BRD v1.3 + integrated project evidence

15.3 Administrator Control Center

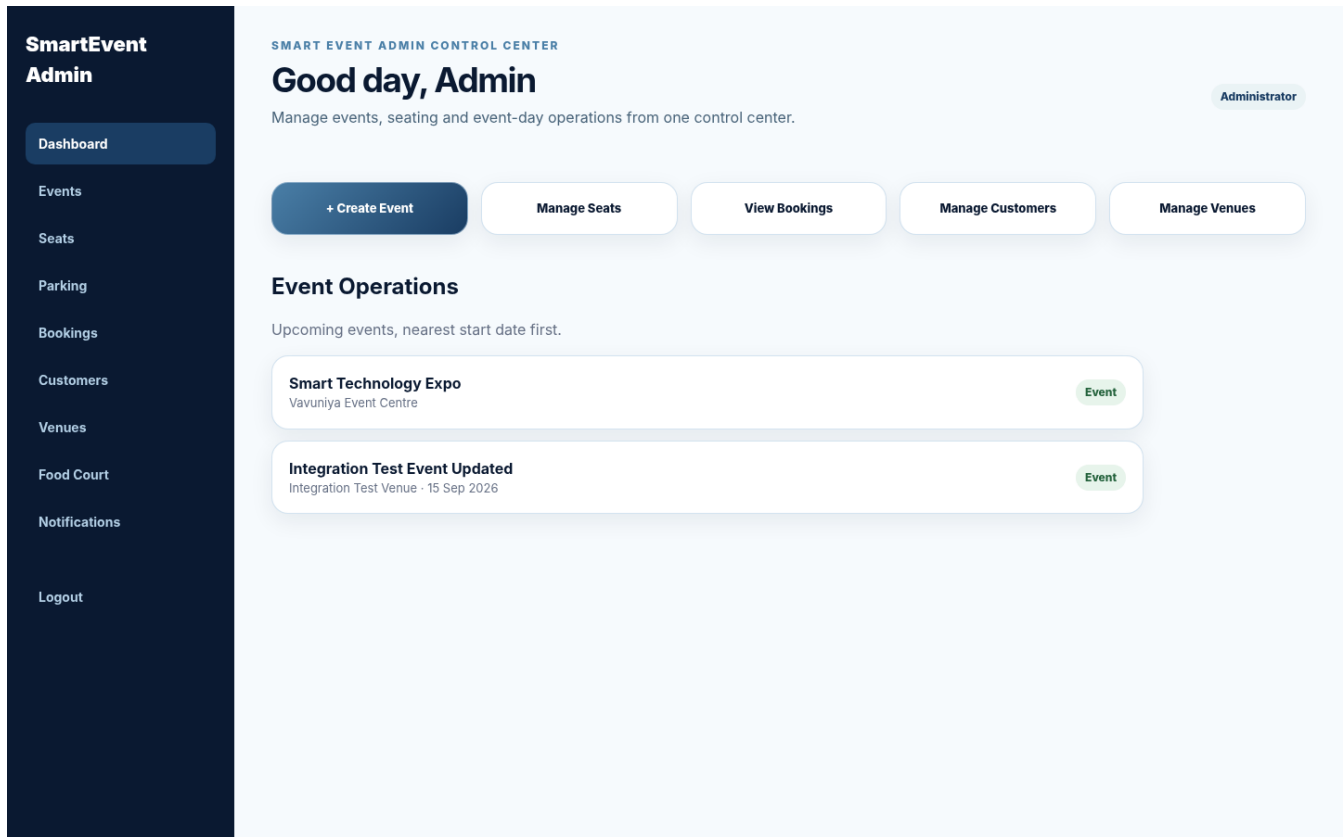


Figure 3 - Administrator Dashboard / Control Center

Technical relevance: the dashboard is a static HTML/CSS/JavaScript page that consumes role-protected APIs through the shared Fetch/JWT frontend helpers. It centralizes links to Events, Seats, Bookings, Customers and Venues without introducing a frontend framework.

15.4 Customer Notification Frontend

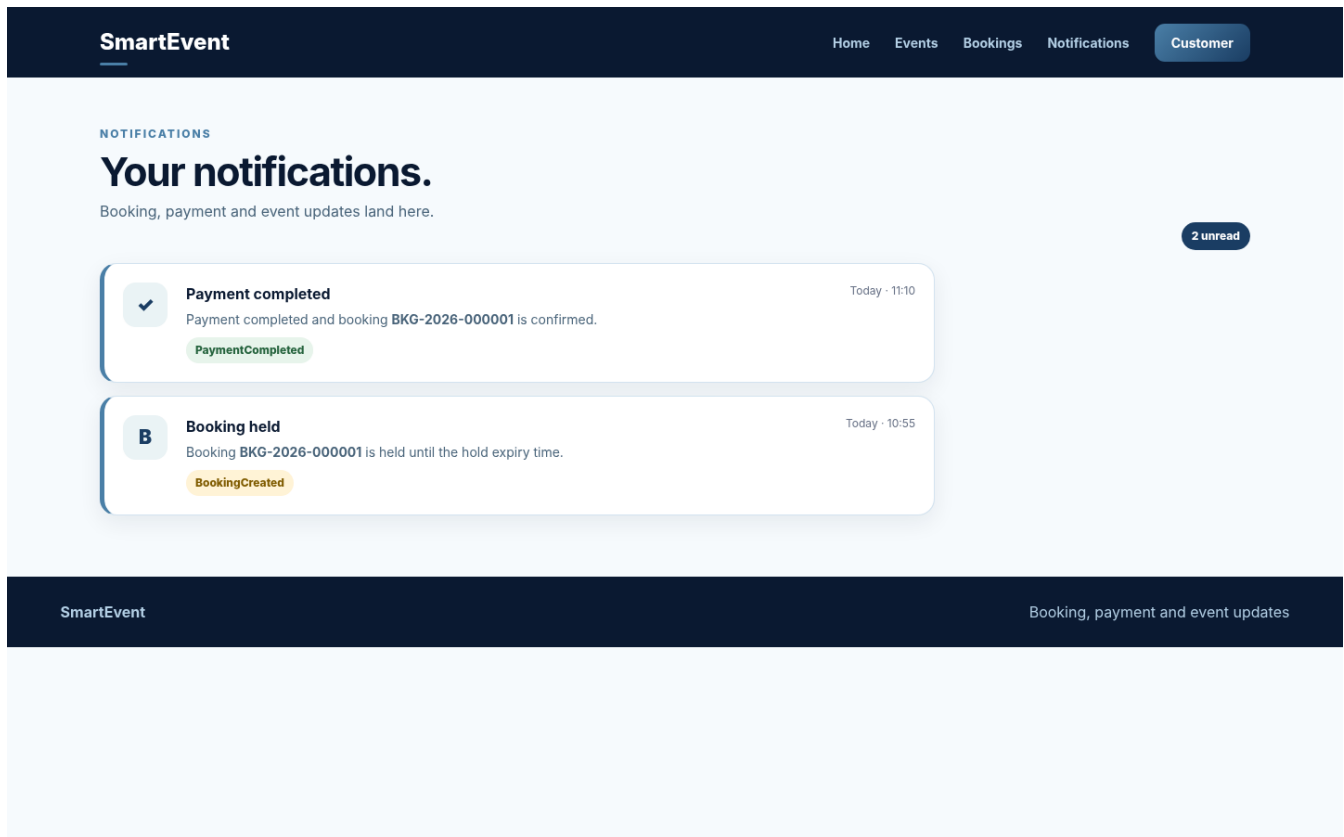


Figure 4 - Customer Notifications Page

Technical relevance: notification records are created by backend business actions and retrieved by the authenticated frontend. This page visually demonstrates JWT-protected Fetch integration and API-driven customer feedback after booking/payment operations.